ML INTERNSHIP PROJECT / FINAL CORRECTED BLUEPRINT

Customer Churn Analytics

One-Day MVP: from CSV validation to a properly evaluated model and an interactive Streamlit application.

Purpose

Build a compact, end-to-end deliverable that demonstrates practical data science judgment: validate the data, quantify segment-level churn, compare baseline and candidate classifiers without test-set leakage, and present results in a three-page dashboard.

10,000	2,037	20.37%	3
customer records	observed exits	overall churn	countries in file

The deliverable to aim for today

A reproducible repository with a data audit, four segment summaries, a cross-validated model comparison, held-out evaluation of the selected model, a functioning Streamlit dashboard, and clear setup instructions. This is a scoped MVP - not a complete banking retention system or causal study.

Build principles

01	**Evidence first** Calculate every KPI and finding from the CSV; never hardcode the displayed statistics.
02	**Honest provenance** Describe the file cautiously; do not claim it is real ECB or proprietary bank data.
03	**Test discipline** Choose the model using training-only cross-validation; keep the test set out of model selection.
04	**Useful, not bloated** Ship the core application before adding SHAP, clustering, a research paper, or extra pages.

Prepared 24 October 2026 | Source file: European_Bank.csv | Replaces the earlier broad-scope plan with a corrected one-day build plan.

1. Dataset reality check

Start by confirming that the file you actually received matches these observed characteristics. If your local copy differs, update the analysis rather than forcing the expected results.

10,000	14	0	0
rows	columns	missing cells	duplicate customer IDs

Audit item	Observed result / implication
Target `Exited`	2,037 exited (20.37%) and 7,963 retained (79.63%).
Duplicate rows	0 exact duplicate rows detected.
Year field	1 unique value(s): 2025. It does not support time-trend analysis and should be excluded from model features.
Feature set	Demographic, geographic, financial, tenure, product and activity fields; no transaction history, campaign history, actual revenue, or customer-level time series.

Data provenance and claims

Use a cautious description in README and presentation

The file resembles the widely circulated educational Churn_Modelling benchmark, but the exact provenance of this specific CSV has not been independently verified. Describe it as an educational customer-churn dataset supplied for this project. Do not present it as actual European Central Bank data or proprietary records from a European bank.

What the data can and cannot support

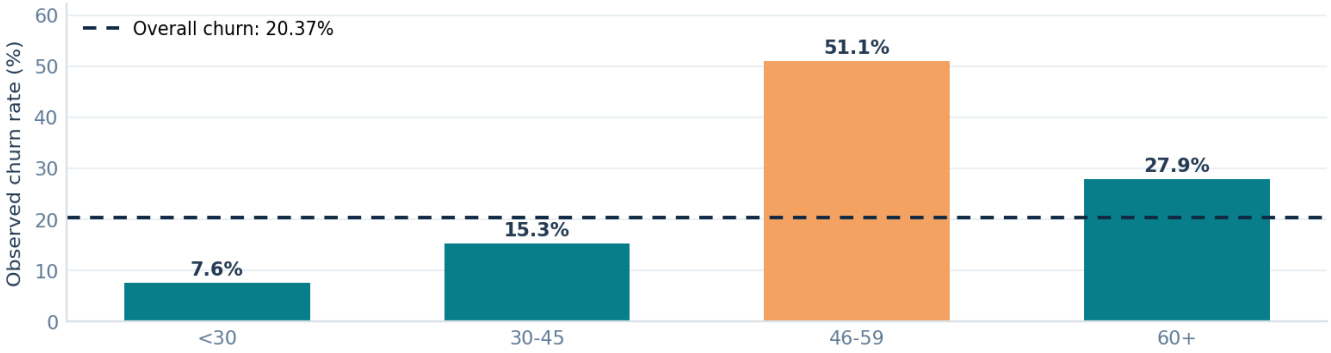
Supported by the listed fields	Not supported without additional data
Descriptive churn differences across the three countries and observed customer segments.	Claims about all European banks or country-level causes of churn.
Associations between churn and the supplied customer attributes.	Proof that inactivity, age, balance, or another feature causes churn.
Retrospective classification of the `Exited` label.	Forecasting when a customer will churn or whether a retention offer would prevent it.
High-balance segment churn patterns as a financial-profile proxy.	Actual revenue loss, customer lifetime value, or return on retention spend.

Quality checks must fail loudly for unexpected schema, missing required columns, non-binary `Exited`, or duplicate customer IDs. Do not silently fill or remove unexpected records without reporting what changed.

2. Segmentation: definitions and verified baseline

These are descriptive rates from the uploaded CSV. They are not evidence of causation. Show both the segment denominator and churn rate so small groups do not appear more reliable than they are.

Observed churn differs materially by age band



Dimension	Segment	Customers (n)	Exited	Churn rate	Share of all exits
Geography	France	5,014	810	16.15%	39.76%
Geography	Germany	2,509	814	32.44%	39.96%
Geography	Spain	2,477	413	16.67%	20.27%
Age band	<30	1,641	124	7.56%	6.09%
Age band	30-45	6,248	956	15.30%	46.93%
Age band	46-59	1,585	810	51.10%	39.76%
Age band	60+	526	147	27.95%	7.22%
Products	1	5,084	1,409	27.71%	69.17%
Products	2	4,590	348	7.58%	17.08%
Products	3 *	266	220	82.71%	10.80%
Products	4 *	60	60	100.00%	2.95%
Engagement	Inactive	4,849	1,302	26.85%	63.92%
Engagement	Active	5,151	735	14.27%	36.08%

Findings to feature carefully

The 46-59 group has 1,585 customers and a 51.10% churn rate, about 2.51 times the overall rate. Germany has a 32.44% observed rate. Product groups marked * have fewer than 300 customers; the 4-product group has only 60 customers, so its 100% rate should be reported with that denominator visible.

* Small-sample marker: product segments with n < 300. “Share of all exits” = exits in segment / 2,037 total exits; it differs from the segment churn rate.

3. Machine-learning design and evaluation

The ML objective is retrospective classification: estimate whether a record is associated with `Exited = 1` using the features available in this dataset. The model is not a causal explanation and is not a validated operational banking risk system.

Feature policy

Role	Fields	Rule
Target only	`Exited`	Never include in X; use only as y.
Exclude from model	`CustomerId`, `Surname`, `Year`	Identifier, personal identifier, and constant year field; keep raw file unchanged for audit.
Categorical candidates	`Geography`, `Gender`, `NumOfProducts`, `HasCrCard`, `IsActiveMember`	Impute if needed; one-hot encode. Treat product count as categorical for the baseline model to avoid imposing a simple linear trend.
Numeric candidates	`CreditScore`, `Age`, `Tenure`, `Balance`, `EstimatedSalary`	Median imputation and scaling for Logistic Regression. Scaling is not required by HistGradientBoosting but is harmless when consistently handled.

Split and model-selection protocol

1. **Split once**
   Stratified 80/20 train/test split with fixed `random_state=42`. Do not examine test performance while choosing settings.
2. **Cross-validate on training data**
   Use 5-fold `StratifiedKFold(shuffle=True, random_state=42)`. Compare candidates by mean Average Precision (AP); report fold mean and standard deviation.
3. **Fit candidates**
   Evaluate `DummyClassifier(strategy="prior")`, `LogisticRegression(max_iter=2000)`, and `HistGradientBoostingClassifier`.
4. **Select without test leakage**
   Select the non-dummy candidate with the best mean training-fold AP. The test set must not determine the winning model.
5. **Final test evaluation**
   Refit the selected pipeline on the full training split and evaluate once on the held-out test split. Evaluate the Dummy baseline on test only as a reference, not as a selection mechanism.

Important implementation detail

One pipeline per estimator

Build a separate sklearn Pipeline for each model, with a ColumnTransformer inside the pipeline. Preprocessing must be fitted inside each training fold during cross-validation. HistGradientBoostingClassifier needs dense input; use `OneHotEncoder(handle_unknown="ignore", sparse_output=False)` with scikit-learn >= 1.2, or configure model-specific preprocessing accordingly.

4. Evaluation outputs and Streamlit design

Use metrics that reflect the approximately 20.37% positive class. Accuracy alone is not adequate: a classifier that predicts “stay” for everyone would appear accurate while missing every churned customer.

Metrics to report

Metric	When / how to use it
Average Precision (AP)	Primary selection metric from cross-validation; report mean and standard deviation. Label it AP rather than loosely calling every PR summary “PR-AUC.”
ROC-AUC	Ranking/discrimination metric, reported for CV and selected-model test evaluation.
Precision, recall, F1	At a declared threshold (0.5 for the simple baseline report). State that this threshold is not automatically the best business operating point.
Confusion matrix	Selected model on held-out test set at threshold 0.5; clearly label positive class as exited.
ROC and precision-recall curves	Plot from saved held-out test predictions; do not refit or regenerate predictions on every Streamlit page load.

Three-page dashboard

Page	Main content	Must-have behavior
1. Overview (`app.py`)	Total customers, exited customers, overall churn; target split; headline segment findings.	KPI values are calculated by shared functions. Avoid hardcoded counts or rates. Include the data-source/provenance note.
2. Segments	Country, age band, product count, active/inactive summaries and charts.	Show n beside every churn rate; flag product groups with n < 300; filters must handle zero-result cases safely.
3. Model Performance	CV model comparison, selected model test metrics, ROC curve, precision-recall curve, confusion matrix.	Read artifacts only. Never retrain on page load. Distinguish CV metrics from held-out test metrics.

Artifacts that the dashboard reads

`artifacts/metrics.json` (CV comparison and final test metrics), `artifacts/model.joblib` (selected fitted pipeline), and `artifacts/test_predictions.csv` (anonymous y_true, probability, and predicted label for held-out curves/confusion matrix). Do not include CustomerId, Surname, or customer-level features in the predictions artifact.

5. Repository structure and module responsibilities

Keep the code modular and small. Avoid a single giant app script that duplicates logic between the dashboard and the training code.

```
european-bank-churn/
├── README.md
├── requirements.txt
├── app.py                    # Overview
├── pages/
│   ├── 1_Segments.py
│   └── 2_Model_Performance.py
├── src/
│   ├── data_validation.py    # schema + data quality checks
│   ├── segmentation.py       # reusable segment tables
│   ├── train.py              # split, CV, fit, save pipeline
│   └── evaluate.py           # held-out metrics and curve data
├── data/
│   └── raw/European_Bank.csv  # local input, if redistribution permitted
└── artifacts/
    ├── model.joblib
    ├── metrics.json
    └── test_predictions.csv
```

Responsibilities and interfaces

Module	Contract
`data_validation.py`	Load CSV; verify required columns, expected target values, duplicate IDs, nulls, category/range anomalies. Return data and an audit summary; never silently mutate the raw CSV.
`segmentation.py`	Expose functions that return count, churned, retained, churn rate, and share of all exits for country, age band, product count, and engagement. Share the same functions with the app.
`train.py`	Create split; cross-validate models using training data only; choose by mean AP; fit chosen model on training partition; save selected pipeline, CV summary, and held-out predictions.
`evaluate.py`	Calculate test metrics, confusion matrix, ROC/PR curve arrays and classification metrics from held-out labels and scores only. Avoid retraining.
Streamlit pages	Use shared audit/segmentation functions or saved artifacts. Page navigation, chart rendering and filters must not trigger model fitting.

Suggested requirements

`pandas`, `numpy`, `scikit-learn>=1.2,<2.0`, `streamlit`, `matplotlib`, `joblib`. Pin tested versions once the app runs successfully. Keep dependency scope small and include a clean setup command in README.

Data sharing note: only commit the CSV to a public repository if the file source and redistribution terms permit it. Otherwise, document the expected local path and keep the data out of version control.

6. One-day execution plan and acceptance checklist

Work in this order. Do not build the visual application before the data and model outputs are reliable. Time boxes are planning aids, not guarantees.

Order	Work block	Done when
1	Inspect data and implement validation	Run logs expected row count, target distribution, null/duplicate checks and schema. Mismatch is reported rather than papered over.
2	Implement four segment summaries	Country, age, products and activity tables match calculations from the CSV; each rate includes its denominator.
3	Build training and evaluation pipeline	Training-only 5-fold CV; choose by AP; fit chosen model on train split; evaluate held-out test once; save artifacts.
4	Implement Overview and Segments	App starts, charts are data-driven, filters work, zero-result states are handled, small product groups are flagged.
5	Implement Model Performance	Shows CV comparison and separate test performance for selected model; confusion matrix and curves match saved test predictions.
6	Smoke-test and document	Fresh environment setup works; pages never refit; README includes run commands, feature policy, metrics meaning, and provenance caveat.

Definition of done

- Data audit asserts required columns, binary target, and unique `CustomerId` values for the current file.
- Every segmentation group has `n`, exited count, churn rate, and contribution to all exits; all group counts sum to 10,000 for the current file.
- The age-band definitions are exactly: <30, 30-45, 46-59, and 60+; the 46-59 rate reproduces from the data, not from a hardcoded constant.
- Preprocessing is inside each estimator Pipeline and is fitted only within training folds / training data.
- Model selection uses cross-validation on training data; the held-out test set is not used to pick the winner.
- Artifacts exist: `model.joblib`, `metrics.json`, `test_predictions.csv`; each contains no customer identifier or surname.
- Three pages open successfully; page load is read-only with respect to model fitting.
- README states what the data can and cannot support; results are described as associations, not causes.

If the schedule slips, cut scope in this order

First omit the Model Performance visual page but still save evaluation artifacts; next omit HistGradientBoosting and ship Dummy + Logistic Regression; then reduce visual polish. Never cut honest provenance, a held-out test split, sample sizes next to churn rates, or the basic smoke test.

7. Copy-paste implementation brief for Antigravity

Paste the instructions below after placing `European_Bank.csv` in the project directory. Keep the agent constrained to the agreed one-day MVP.

Build a reproducible customer-churn analytics MVP from European_Bank.csv. Inspect the actual schema first and report any mismatch; do not hardcode counts or rates.
SCOPE
1) src/data_validation.py: validate required fields, target Exited in {0,1}, duplicate CustomerId, missing values, and relevant ranges. Preserve raw data; exclude CustomerId, Surname and Year from model features.
2) src/segmentation.py: reusable summary functions for Geography, age bands (<30, 30-45, 46-59, 60+), NumOfProducts, and IsActiveMember. Return n, exited, retained, churn_rate and share_of_all_exits.
3) src/train.py: stratified 80/20 split (random_state=42); use 5-fold StratifiedKFold on training data; compare DummyClassifier(strategy="prior"), LogisticRegression(max_iter=2000), and HistGradientBoostingClassifier. Use mean cross-validated Average Precision (AP) to select the best non-dummy model. Each estimator must have its own sklearn Pipeline with a ColumnTransformer. Fit preprocessing within each training fold. Use dense one-hot output for HistGradientBoosting compatibility. Fit the selected pipeline on the training partition only.
4) src/evaluate.py: evaluate the selected model once on the held-out test split. Save CV model comparison, selected-model test metrics, Dummy test baseline, confusion matrix at threshold 0.5, and ROC/precision-recall curve data. Save the selected fitted pipeline to artifacts/model.joblib, metrics to artifacts/metrics.json, and anonymous y_true / probability / y_pred rows to artifacts/test_predictions.csv. Do not include IDs, surname or customer features in the prediction artifact.
5) Streamlit: app.py is Overview; pages/1_Segments.py; pages/2_Model_Performance.py. Exactly three pages. Pages load saved artifacts and must never train/refit a model on page load. Use shared segmentation functions; show n beside every rate; flag segment groups with n < 300.
6) README.md and requirements.txt: setup/run commands, module descriptions, model evaluation protocol, limitation notes, and dataset provenance statement. Do not claim real ECB or bank-proprietary data; exact provenance of this CSV is not independently verified.
QUALITY GATES
- Never use Exited as a feature.
- Never use held-out test performance to select/tune the winning model.
- Label Average Precision (AP) clearly; do not rely on accuracy alone.
- Include tests for segment totals, zero-result filters, artifact presence, and app startup.
- Avoid SHAP, clustering, CatBoost, SMOTE, fairness audits, research paper, executive summary, and additional dashboard pages until the MVP works end to end.

Out of scope for the one-day MVP

SHAP explanations, clustering, CatBoost, SMOTE, calibration analysis, formal subgroup fairness audits, risk-explorer tools, causal inference, a research paper, a polished executive summary, and actual revenue-at-risk estimates. Add these only if the core application is working and the internship requirements justify them.

Final limitation statement

What this project demonstrates

It demonstrates data validation, segmentation analytics, a carefully evaluated retrospective classifier, and an interactive reporting interface. It does not prove causality, forecast churn timing, establish bank-wide representativeness, measure actual revenue loss, or demonstrate that retention interventions work. Those claims require additional data and validation.

Reference note: the supplied file has 10,000 rows, 14 columns, no missing cells, no duplicate rows, no duplicate CustomerId values, 2,037 exits, and a constant Year=2025. All numerical results in this blueprint are calculated directly from /mnt/data/European_Bank.csv as supplied for this project.